МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙГОСУДАРСТВЕННЫЙТЕХНИЧЕСКИЙУНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил:
А.С. Мышкевич
студент группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
05.03.2026

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1:

6) Множество вещественных чисел ограниченной мощности – Предусмотреть возможность пересечения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Выполнение:

Код программы

```
class LimitedSet:
    def __init__(self, capacity, elements=None):
        if capacity <= 0:
            raise ValueError("Мощность должна быть положительной")

        self.capacity = capacity
        self.elements = []

        if elements:
            for elem in elements:
                if len(self.elements) < self.capacity:
                    if float(elem) not in self.elements:
                        self.elements.append(float(elem))

    def __str__(self):
        return f"{{{', '.join(str(x) for x in self.elements)}}}"

    def __eq__(self, other):
        if not isinstance(other, LimitedSet):
            return False
        return set(self.elements) == set(other.elements)

    def contains(self, value):
        return float(value) in self.elements

    def add(self, value):
        try:
            num = float(value)
```

```
            if num in self.elements:
                print(f"Элемент {num} уже существует")
            elif len(self.elements) >= self.capacity:
                print(f"Нельзя добавить {num}: достигнута мощность {self.capacity}")
            else:
                self.elements.append(num)
        except ValueError:
            print(f"Ошибка: {value} не является числом")

    def remove(self, value):
        try:
            self.elements.remove(float(value))
        except ValueError:
            print(f"Элемент {value} не найден")
        except (TypeError, ValueError):
            print(f"Ошибка: {value} не является числом")

    def intersection(self, other):
        common = [x for x in self.elements if x in other.elements]
        return LimitedSet(len(common), common)


# Пример использования
set1 = LimitedSet(5, [1.5, 2.7, 3.14, 2.7])  # Дубль 2.7 не добавится
set2 = LimitedSet(5, [2.7, 3.14, 5.5])

print("Множество 1:", set1)
print("Множество 2:", set2)

set1.add(4.8)
print("После добавления 4.8:", set1)

set1.add(4.8)  # Попытка добавить дубль
set1.add(6.6)  # Попытка добавить при полной мощности (если 5 элементов)

print("Содержит 2.7?", set1.contains(2.7))
print("Содержит 9.9?", set1.contains(9.9))

set1.remove(2.7)
print("После удаления 2.7:", set1)

set3 = set1.intersection(set2)
print("Пересечение множеств:", set3)

set4 = LimitedSet(5, [4.8, 1.5, 3.14])
print("Множество 4:", set4)
print("set1 == set4?", set1 == set4)
```

Рисунки с результатами работы программы

```
Множество 1: {1.5, 2.7, 3.14}
Множество 2: {2.7, 3.14, 5.5}
После добавления 4.8: {1.5, 2.7, 3.14, 4.8}
Элемент 4.8 уже существует
Содержит 2.7? True
Содержит 9.9? False
После удаления 2.7: {1.5, 3.14, 4.8, 6.6}
Пересечение множеств: {3.14}
Множество 4: {4.8, 1.5, 3.14}
set1 == set4? False
```

Задание 2:

6) Система Телефонная станция. Абонент оплачивает Счет за разговоры и Услуги, может попросить Администратора сменить номер и отказаться от услуг. Администратор изменяет номер, Услуги и временно отключает Абонента за неуплату.

Выполнение:

Код программы

```
from abc import ABC, abstractmethod
from datetime import datetime
from typing import List, Optional


# ========== БАЗОВЫЕ КЛАССЫ ==========
class Bill(ABC):
    """Абстрактный счет"""

    def __init__(self, amount=0):
        self.amount = amount
        self.date = datetime.now()
        self.paid = False

    @abstractmethod
    def calculate(self):
        pass

    def pay(self):
        self.paid = True
        print(f"Счет оплачен: {self.calculate()} руб.")

    def __str__(self):
        return f"Счет от {self.date.strftime('%d.%m.%Y')}: {self.calculate()} руб."


class CallBill(Bill):
    """Счет за разговоры"""
```

```
    def __init__(self, minutes, rate):
        super().__init__()
        self.minutes = minutes
        self.rate = rate

    def calculate(self):
        return self.minutes * self.rate


class ServiceBill(Bill):
    """Счет за услуги"""

    def __init__(self, service_name, monthly_fee):
        super().__init__()
        self.service_name = service_name
        self.monthly_fee = monthly_fee

    def calculate(self):
        return self.monthly_fee


class Service:
    """Услуга"""

    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} ({self.price} руб./мес)"


# ========== ОСНОВНЫЕ КЛАССЫ ==========
class Subscriber:
    """Абонент"""

    def __init__(self, sub_id, name, phone):
        self.id = sub_id
        self.name = name
        self.phone = phone
        self.balance = 0.0
        self.active = True
        self.services = []  # агрегация
        self.bills = []  # ассоциация

    def add_service(self, service):
        if service not in self.services:
            self.services.append(service)
            bill = ServiceBill(service.name, service.price)
```

```
        self.bills.append(bill)
        print(f"{self.name} подключил {service.name}")

    def remove_service(self, service_name):
        for s in self.services[:]:
            if s.name == service_name:
                self.services.remove(s)
                print(f"{self.name} отключил {service_name}")
                return

    def make_call(self, minutes, rate):
        if not self.active:
            print(f"{self.name} заблокирован. Звонок невозможен")
            return
        bill = CallBill(minutes, rate)
        self.bills.append(bill)
        print(f"{self.name} проговорил {minutes} мин")

    def pay_bill(self, index=-1):
        if not self.bills:
            print("Нет счетов")
            return
        bill = self.bills[index] if index != -1 else self.bills[-1]
        amount = bill.calculate()
        bill.pay()
        self.balance -= amount

    def get_debt(self):
        return sum(b.calculate() for b in self.bills if not b.paid)

    def request_new_phone(self, admin, new_phone):
        print(f"{self.name} просит сменить номер {self.phone} -> {new_phone}")
        return admin.change_phone(self, new_phone)

    def request_unsubscribe(self, admin, service_name):
        print(f"{self.name} просит отключить {service_name}")
        return admin.remove_service(self, service_name)

    def __str__(self):
        status = "активен" if self.active else "заблокирован"
        serv = ", ".join(s.name for s in self.services) or "нет"
        return f"{self.name} (тел:{self.phone}, {status}, услуги:{serv})"

    def __eq__(self, other):
        return isinstance(other, Subscriber) and self.id == other.id


class Administrator:
    """Администратор"""
```

```
    def __init__(self, name, station):
        self.name = name
        self.station = station  # ассоциация

    def change_phone(self, subscriber, new_phone):
        # Проверка, что номер не занят
        for sub in self.station.subscribers:
            if sub.phone == new_phone:
                print(f"Ошибка: номер {new_phone} занят")
                return False
        old = subscriber.phone
        subscriber.phone = new_phone
        print(f"Админ {self.name} сменил номер {old} -> {new_phone}")
        return True

    def add_service(self, subscriber, service_name):
        service = self.station.get_service(service_name)
        if service:
            subscriber.add_service(service)
            return True
        print(f"Услуга {service_name} не найдена")
        return False

    def remove_service(self, subscriber, service_name):
        subscriber.remove_service(service_name)
        return True

    def block_subscriber(self, subscriber):
        if subscriber.get_debt() > 0:
            subscriber.active = False
            print(f"Админ {self.name} заблокировал {subscriber.name} за долг
{subscriber.get_debt()} руб.")
            return True
        print(f"У {subscriber.name} нет долгов")
        return False

    def unblock_subscriber(self, subscriber):
        if subscriber.get_debt() == 0:
            subscriber.active = True
            print(f"Админ {self.name} разблокировал {subscriber.name}")
            return True
        print(f"{subscriber.name} должен {subscriber.get_debt()} руб.")
        return False

    def __str__(self):
        return f"Админ {self.name}"


class PhoneStation:
    """Телефонная станция (агрегация)"""
```

```
    def __init__(self, name, call_rate=2.0):
        self.name = name
        self.call_rate = call_rate
        self.subscribers = []  # агрегация
        self.services = []  # агрегация

    def add_subscriber(self, subscriber):
        if subscriber not in self.subscribers:
            self.subscribers.append(subscriber)
            print(f"{subscriber.name} добавлен на станцию")

    def remove_subscriber(self, sub_id):
        for s in self.subscribers[:]:
            if s.id == sub_id:
                self.subscribers.remove(s)
                print(f"{s.name} удален со станции")
                return

    def add_service(self, service):
        self.services.append(service)
        print(f"Услуга {service.name} добавлена")

    def get_service(self, name):
        for s in self.services:
            if s.name == name:
                return s
        return None

    def show_all(self):
        print(f"\n=== СТАНЦИЯ '{self.name}' ===")
        print("УСЛУГИ:")
        for s in self.services:
            print(f"  - {s}")
        print("АБОНЕНТЫ:")
        for s in self.subscribers:
            debt = s.get_debt()
            print(f"  - {s}" + (f" ДОЛГ: {debt} руб." if debt else ""))

    def __str__(self):
        return f"Станция '{self.name}' ({len(self.subscribers)} абонентов)"


# ========== ДЕМОНСТРАЦИЯ ==========
def demo():
    print("-" * 50)
    print("ТЕЛЕФОННАЯ СТАНЦИЯ")
    print("-" * 50)

    # 1. Создаем станцию
```

```
station = PhoneStation("Мегафон", 1.5)
admin = Administrator("Петров", station)
print(f"Станция: {station}")
print(f"Админ: {admin}")

# 2. Добавляем услуги
print("\n--- Добавление услуг ---")
station.add_service(Service("Интернет", 300))
station.add_service(Service("ТВ", 200))
station.add_service(Service("SMS", 150))

# 3. Создаем абонентов
print("\n--- Регистрация абонентов ---")
sub1 = Subscriber("001", "Иванов Иван", "111-111")
sub2 = Subscriber("002", "Петров Петр", "222-222")
sub3 = Subscriber("003", "Сидорова Анна", "333-333")

station.add_subscriber(sub1)
station.add_subscriber(sub2)
station.add_subscriber(sub3)

# 4. Подключаем услуги
print("\n--- Подключение услуг ---")
admin.add_service(sub1, "Интернет")
admin.add_service(sub1, "ТВ")
admin.add_service(sub2, "SMS")

# 5. Совершаем звонки
print("\n--- Звонки ---")
sub1.make_call(10, station.call_rate)
sub1.make_call(25, station.call_rate)
sub2.make_call(5, station.call_rate)

# 6. Смена номера
print("\n--- Смена номера ---")
sub1.request_new_phone(admin, "999-999")

# 7. Проверка счетов
print("\n--- Счета ---")
for i, bill in enumerate(sub1.bills):
    print(f"  {i+1}. {bill}")

# 8. Оплата
print("\n--- Оплата ---")
sub1.pay_bill(0)
sub1.pay_bill()

# 9. Блокировка за долги
print("\n--- Блокировка ---")
print(f"Долг {sub2.name}: {sub2.get_debt()} руб.")
```

```
    admin.block_subscriber(sub2)

    # Попытка звонка с заблокированного
    sub2.make_call(1, station.call_rate)

    # 10. Отказ от услуги
    print("\n--- Отказ от услуги ---")
    sub1.request_unsubscribe(admin, "ТВ")

    # 11. Показываем состояние
    station.show_all()

    # 12. Сравнение
    print("\n--- Сравнение ---")
    sub1_copy = Subscriber("001", "Иванов Иван", "999-999")
    print(f"sub1 == sub1_copy? {sub1 == sub1_copy}")


if __name__ == "__main__":
    demo()
```

Рисунки с результатами работы программы

```
------------------------------------------------
ТЕЛЕФОННАЯ СТАНЦИЯ
------------------------------------------------
Станция: Станция 'Мегафон' (0 абонентов)
Админ: Админ Петров

--- Добавление услуг ---
Услуга Интернет добавлена
Услуга ТВ добавлена
Услуга SMS добавлена

--- Регистрация абонентов ---
Иванов Иван добавлен на станцию
Петров Петр добавлен на станцию
Сидорова Анна добавлен на станцию

--- Подключение услуг ---
Иванов Иван подключил Интернет
Иванов Иван подключил ТВ
Петров Петр подключил SMS

--- Звонки ---
Иванов Иван проговорил 10 мин
Иванов Иван проговорил 25 мин
Петров Петр проговорил 5 мин

--- Смена номера ---
Иванов Иван просит сменить номер 111-111 -> 999-999
Админ Петров сменил номер 111-111 -> 999-999
```

Вывод: закрепил навыки объектно-ориентированного программирования на языке Python.